Jewellery QR Composition, Cloud Inventory & Dynamic Pricing System – Requirements Specification

Version: 4.0

Status: Approved Requirements

Owner: User

1. PURPOSE

This document specifies the requirements for a jewellery system that:

- Stores all jewellery item data in a central cloud-accessible database.
- Generates QR codes for each jewellery item.
- Encodes only composition data + SKU inside the QR (no prices).
- Automatically generates a unique SKU for each item.
- On the SCAN side, computes the CURRENT price using configurable rates when an item is scanned.

Key concepts:

- CREATE SIDE: Used by staff to create items, auto-generate SKU, store in cloud DB, and generate QR codes.
- SCAN SIDE: Used at counter / sales to scan QR, fetch composition, input current rates, and compute current price.

The QR itself must NOT contain any price or rate, only:

- SKU (unique, auto-generated by system)
- Gold purity
- Gold weight (grams)
- Up to six (6) diamond types, each with its own weight (carats)
- Optional item name/description

The SCAN view is responsible for price calculation using CURRENT rates at time of sale.

2. SCOPE

In Scope:

- Auto-generated unique SKU for each new item.
- Web-based CREATE interface for item entry and QR generation.
- Web-based SCAN interface for reading QR and computing price.

- Cloud database to store all item records.
- API endpoints to create/read items and generate QR images.
- Dynamic price computation on SCAN side based on current operator-entered rates.
- Dockerized deployment of backend and frontend.

Out of Scope:

- Automatic fetching of gold/diamond rates from external APIs (MVP assumes manual rate entry).
- Complex tax rules beyond simple percentage.
- User authentication and role management.
- Digital signatures for tamper-proof QRs (future enhancement).
- POS/invoice printing (billing system consumes the computed values separately).

3. USERS & USE CASES

3.1 User Types

- Staff (Create Side):
- Enters jewellery composition.
- Relies on auto-generated SKU.
- Generates QR codes and stores items into the cloud database.
- Salesperson / Manager (Scan Side):
- Scans QR.
- Enters current gold and diamond rates.
- Uses computed current price to quote to customers or pass to billing.
- Billing Operator:
- Uses composition + computed price to produce bill in a separate billing system.
- Customer (Optional):
- May scan QR to view composition and price if you expose the SCAN interface.

3.2 Use Cases

UC-01: Create New Jewellery Item & Generate QR

1. Staff opens the CREATE page.
2. Staff enters:
- Item name.

- Gold purity (e.g. 18K, 22K).

- Gold weight in grams.

- Up to 6 diamond entries, each with:

- diamond type (text description)

- diamond weight in carats.

3. System automatically generates a unique SKU (e.g. numeric sequence, or prefixed code).

4. Staff can optionally override SKU (if allowed), but uniqueness must be enforced.

5. System validates all inputs.

6. System constructs a JSON payload that will be embedded in the QR.

7. System stores the full item record in the cloud database, including:

- auto-generated SKU

- composition fields

- timestamps.

8. System calls QR generator backend to create a PNG QR image encoding the JSON payload.

9. The QR image is displayed; staff can download/print and attach the QR label to the jewellery.

UC-02: Scan QR and Compute Current Price

1. Salesperson opens the SCAN page on a device (phone/tablet/PC).

2. The SCAN page activates the camera and starts QR scanning.

3. Salesperson points camera at the jewellery QR.

4. The system decodes the QR and parses the JSON payload.

5. The system extracts composition:

- SKU

- Name (if present)

- Gold purity

- Gold weight (grams)

- Diamonds list with type + weight (carats).

6. The system displays the composition to the user.

7. The system optionally uses SKU to fetch the latest item data from the cloud database (e.g. to handle any manual edits).

8. On the same SCAN page, there is a PRICE COMPUTATION panel where the user enters:

- Current gold rate per gram (for the given purity).

- Current diamond rate(s):

- Either a single diamond rate per carat,

- Or per-type rates (e.g. separate rate fields matching each diamond type line).

- Making charges:

- Either a flat amount or per gram (MVP: flat amount).

- Tax percent.

9. The system computes:

- Gold value = gold_weight_g * gold_rate_per_g.

- Diamond value = sum(diamond_weight_ct_i * rate_per_ct_i).

- Subtotal = gold value + diamond value + making_charges.

- Tax = subtotal * (tax_percent / 100).

- Final price = subtotal + tax.

10. The system displays a clear price breakdown and final current price.

11. Salesperson can communicate this price to customer or use it in billing.

UC-03: Retrieve Item from Cloud by SKU (Without QR)

1. User accesses an item list/search interface (future minimal view).

2. User searches by SKU or name.

3. System returns stored item composition from database.

4. User can re-generate QR or compute price manually as in SCAN use case.

4. FUNCTIONAL REQUIREMENTS

4.1 QR Payload Schema

Each QR code must encode exactly one JSON object with the following structure:

{

"sku": "RING-000001",

"name": "18K Diamond Ring",

"purity": "18K",

"gold_weight_g": 5.80,

"diamonds": [

{ "type": "Center Stone VVS EF", "weight_ct": 0.40 },

{ "type": "Side Stones SI IJ", "weight_ct": 0.25 }

]

}

Field definitions:

- sku (string, required): Auto-generated unique identifier. Example: "RING-000001".
- name (string, optional but recommended): Human-readable item name.
- purity (string, required): Gold purity (e.g. "18K", "22K").
- gold_weight_g (number, required): Gold weight in grams. Must be > 0.
- diamonds (array, required, can be empty): List of up to 6 diamond entries.

Each element in diamonds array:

- type (string, required): Description of diamond type/grade (e.g. "VVS EF", "SI IJ", "Solitaires").
- weight_ct (number, required): Diamond weight in carats. Must be > 0.

Constraints:

- Max 6 diamond entries per item.
- All numeric fields > 0 where applicable.
- No prices, rates, taxes, or making charges in the QR payload.
- JSON must be compact/minified for QR size efficiency.

4.2 Auto-Generated SKU Requirements

- System must generate a SKU automatically on item creation.
- SKU must be unique across all items.
- SKU format may be:
- Numeric sequence with padding (e.g. "000001", "000002"), or
- Prefixed pattern (e.g. "RING-000001", "ITEM-000001").
- The format must be consistent once chosen.
- Uniqueness must be enforced at the database level.
- Optional: allow staff to override SKU at creation time, but system must reject duplicates.

4.3 CREATE PAGE Requirements

Inputs:

- Name (text input).

- Gold purity (dropdown or text input; options like 18K, 22K).

- Gold weight in grams (numeric).

- Diamond lines (up to 6 rows):

- Diamond type (text).

- Diamond weight in carats (numeric).

- SKU field:

- Read-only, auto-filled by system once form is opened or once "Prepare SKU" is clicked.

- Optional ability to edit if you choose (with uniqueness validation).

Behavior:

1. When CREATE page loads, system requests the next SKU or generates it locally (depending on design).

2. User fills form fields.

3. On submit:

- Validate required fields.

- Ensure numeric fields are positive numbers.

- Ensure that for used diamond rows, both type and weight_ct are provided.

- Ensure diamonds length <= 6.

4. Construct QR JSON payload as per schema.

5. Send POST /items to backend to store item in database.

6. On success, send QR payload to POST /api/qr.

7. Display resulting QR image with instruction to save/print.

8. Show a summary of item data on the page for the user to review.

4.4 SCAN PAGE Requirements

Display sections:

- Camera preview & QR scanning.

- Composition display.

- Price computation panel.

- Raw JSON display for debugging.

Steps:

1. On page load, initialize QR scanner and request camera permission.

2. When a QR is detected:

- Decode QR text.

- Parse as JSON.

- If parsing fails, show "Invalid QR payload".

- If parsing succeeds, validate that required fields exist (SKU, purity, gold_weight_g).

3. Populate composition display with:

- SKU.

- Name (if present).

- Purity.

- Gold weight (g).

- Diamond entries: "type – weight_ct ct".

4. Price Computation Panel:

Inputs:

- Gold rate per gram (numeric).

- For diamond pricing, either:

- A single "Diamond rate per carat" (applied to all diamonds), OR

- A list of rates per diamond entry (one input per row; MVP may choose the single-rate approach to keep simple).

- Making charge (flat numeric).

- Tax percent (numeric, e.g., 3).

Behavior:

- When inputs change or a "Compute Price" button is clicked:

- Compute:

- gold_value = gold_weight_g * gold_rate_per_g.

- diamond_value = sum(weight_ct_i * diamond_rate_per_ct_i).

- subtotal = gold_value + diamond_value + making_charge.

- tax = subtotal * (tax_percent / 100).

- final_price = subtotal + tax.

- Display breakdown:

- Gold value.

- Diamond value.

- Making charges.

- Tax amount.

- Final price (rounded to nearest currency unit or two decimals).

5. Optional – DB lookup:

- After scan, system uses SKU to call GET /items/{sku}.

- If cloud data differs from QR (QR might be older), show the mismatch or prefer DB values as canonical.

4.5 CLOUD DATABASE Requirements

Logical schema (PostgreSQL-style):

TABLE items:

- id: uuid primary key.

- sku: text unique not null.

- name: text.

- purity: text not null.

- gold_weight_g: numeric not null.

- diamonds_json: json/jsonb not null.

- created_at: timestamp with time zone default now().

- updated_at: timestamp with time zone default now().

diamonds_json must store an array of diamond objects:

[{ "type": "...", "weight_ct": 0.XX }, ...]

The database must:

- Enforce unique SKU.

- Allow retrieval by SKU quickly (index on sku).

- Be hosted in the cloud (Supabase, Railway Postgres, etc.).

- Be reachable securely from backend.

4.6 API Requirements

Minimum endpoints:

POST /items

- Request body: full item object (no prices):

{ "name": "...", "purity": "...", "gold_weight_g": ..., "diamonds": [...] }

- Backend:

- Assigns/generates SKU (if not already generated client-side).

- Stores record in DB.

- Returns stored item, including SKU.

GET /items/{sku}

- Returns the stored item by SKU.

POST /api/qr

- Request body: QR payload JSON (schema as defined).

- Returns: QR image (PNG).

GET /

- Returns SCAN page.

GET /generate

- Returns CREATE page.

4.7 DOCKER Requirements

Dockerfile must:

- Use a Python 3 base image.

- Install dependencies (e.g., FastAPI, uvicorn, QR code lib, DB client).

- Copy application source code.

- Expose port 8000.

- Start the FastAPI server via uvicorn main:app.

The resulting container must be runnable with:

docker run -p 8000:8000 jewellery-qr-pricing

5. NON-FUNCTIONAL REQUIREMENTS

5.1 Performance

- QR generation within 1 second for typical payloads.
- QR scanning latency bounded primarily by device and JS library speed.
- Price computation should be instantaneous after rate entry.

5.2 Security

- No prices or sensitive financial data in QR.
- Communication between backend and DB should use secure connections (TLS) in production.
- QR spoofing is possible; digital signatures can be added later.

5.3 Reliability

- System must handle invalid / non-JSON QR codes gracefully.
- System must handle missing fields gracefully with clear messages.
- If DB is unreachable, SCAN page should still show QR payload composition but skip DB refresh.

5.4 Portability

- Frontend must work on modern browsers (Chrome, Firefox, Edge, Safari).
- SCAN must work on mobile devices with cameras.
- Docker container must run on Linux/Windows/macOS with Docker installed.

6. FUTURE ENHANCEMENTS

- Integrate external gold/diamond rate APIs to auto-fill rates.
- Add user accounts and roles (staff, admin).
- Implement digital signature or HMAC over QR payload to detect tampering.
- Add inventory dashboards (stock count, valuation).
- Integrate directly with billing / POS systems.
- Automatic label printing formatting for specific printers.
- Multi-branch support with branch IDs on items.

7. SUMMARY

This system provides:

- Auto-generated unique SKU per jewellery item.

- QR codes encoding composition data (purity, gold weight, up to 6 diamond types and weights, SKU, name).

- Cloud storage of all items, accessible from anywhere.

- A SCAN interface that computes CURRENT price at time of sale using entered rates, without embedding prices in QR.

- Dockerized backend and web UI for easy deployment and sharing.

End of Document.